Muhammad Adil Saeed.
24k-0705. || BCS-3J.

COAL Assignment 2.

Question 1

```
mov eax, 3      ; EAX = 3, EBX = --, Stack = [].
mov ebx, 7      ; EAX = 3, EBX = 7, Stack = [].
push eax.       ; Stack = [3].
push ebx.       ; stack = [3, 7].
pop eax.        ; EAX = 7 ; EBX = 7; Stack = [3].
pop push 10     ; Stack = [3, 10]
    pop ebx     ; EAX = 7, EBX = 10, Stack = [3]
    pop eax.    ; EAX = 3, EBX = 10, Stack = [] /empty.
```

Question 2.

```
if (num1 > num2 && num3 != num4)
        result = num1 + num2.
else if (num2 == num4)
        result = num2 - num1.
else result = 0.
```

```
.data
    num1 SDWORD ?
    num2 SDWORD ?
    num3 SDWORD ?
    num4 SDWORD ?
    result SDWORD ?
```

```
.data
  ~~num1~~
.code
  main PROC
    mov eax, num1
    mov ebx, num2
    mov ecx, num3
    mov edx, num4.

;Condition 1.
  cmp eax, ebx.     ; num1 > num2.
  jle CHECKSEC      ; num1 <= num2.

  cmp ecx, edx.     ; num3 != num4.
  je. CHECKSEC.     ; num3 == num4.

; Condition True as of Num1 > num2 & num3 != num4.
    add eax, ecx.
    mov result, eax.
    jum END-IF.
```

```
CHECKSEC :
    ; num 2 == num4 .
    cmp ebx, edx .
    jne ELSE_case .  ; num2 != num 4 .

    sub ebx, eax .      ; num 2 = num4 .
    mov result, ebx .

ELSE_case :
    mov result = 0

END_IF :
    exit
    main ENDP
    END main
```

Question 3

```
Som = 0 ; i = 0 .
while (i<count){,
        if array[i] > 0 .
      Sum += array[i] .
        else
        Sum = Sum - 1 ;
          i = i+1 ;
```

```
.data .
    array   SDWORD  10, -4, 7, 0, 2.
    Count   SDWORD  5 .
    Sum     SDWORD  ?
    i       SDWORD  ?
```

```
.code .
Main PROC .
    mov DWORD PTR sum, 0 .
    mov DWORD PTR i, 0 .

While_LOOP :
    mov eax, count .
    cmp eax, count
    jge END-LOOP ; if i>=count .

    mov eax, i
    mov ebx, array[eax*4]
    cmp ebx, 0  ; if array[i] > 0 .
    jle ELSE_PART ;  " <= 0 .

    mov eax, sum
    add eax, ebx .
    mov sum, eax .

    jmp UPDATE_D

ELSE_PART
    mov eax, sum
    sub eax, 1
    mov sum, eax .

UPDATE_D
    ; i = i+1
    mov eax, i
    inc eax
    mov i, eax .
    jump WHILE_LOOP

END_LOOP
    exit
    main ENDP .

END main .
```

Question 4:

```
Mov AX,0h.
Mov ECX,0Ah.
DOLOOP:
  DEC AX
  LOOP DOLOOP.
```

1. iteration = -1.
2. = -2
3. = -3
4. -4
5. -5
6. -6
7. -7
8. -8
9. -9
10 -10.

AX = -10.

-10 = FFF6h.

$\boxed{AX = FFF6h. [-10]}$.

Loop runs 10 times till ecx = 0.

Question 5:

```
MOV AL,0D4h .; AL= 0D4h.
SHL AL,3.      ; AL=A0h CF=0.

MOV AL,D4h
SAL AL,3       ;AL=~~A0~~ A0h, CF=0

STC ; CF=1.

MOV AL,0D4h.
ROL AL,1       ; AL=A9h .CF=1.

STC.  ;CF=1.

MOV AL,D4h
RCR AL,3       ; CF=~~0~~1, AL=3
```

1101 0100.

1. CF=1 1010 1000
2. CF=1 0101 0000.
3. CF=0 1010 0000.

1010 1001.
CF=1 ~~0101~~ 1

1101 0100. CF=1.
1. 1110 1010 CF=0.
2. 0111 0101

1. 1110 1010 0.
2. 0111 0101 0.
3. 0011 1010 1.

Question 6:

```
-data.
    op1 word ?
    op2 word ?
    x   word ?
    y   word ?
    z   word ?

-code.
Main PROC:
    ; code of given function.

WhileLoop:
    mov ax,op1.
    Cmp ax,op2.
    jg  ENDLOOP.  ; op1 > op2.

; if condition.
    mov ax,op1
    Cmp ax,x.
    jL  ELSEPART ; op1 < x.

; continued....
    cmp ax,y
    jg  ELSEPART  ; if op1>y
Then Part:
    add z,10
    jmp ContinueLoop.
```

```
ELSEPART :
        sub z,10
Continueloop :
        dec op1
        jmp StartLoop
ENDLOOP :
        exit
        main ENDP
        END main
```

Question 7 :

```
INCLUDE Irvine32.inc.
.data
posMsg  BYTE  "Positive",0
negMsg  BYTE  "Negative",0
zeroMsg BYTE  "Zero",0.

.code
main PROC.
        mov al, -5
        cmp al, 0
        je  ZeroCase ; -5-0=0?
        jg  PositiveCase ; -5-0>0?

Negative :
        mov edx,OFFSET negMsg
        call WriteString
        jmp ExitLabel.

Positive :
        move edx,OFFSET POSMsg
        call WriteString
        jmp ExitLabel

Zero :
        move edx, OFFSET zeroMsg
        call WriteString.

ExitLabel :
        call crlf
        exit
main ENDP
  END main.
```

Question 8:

```
Include Irvine32.inc.

.data
    num1 SWORD 300
    num2 SWORD -20.

    result SDWORD ?

.code
  main PROC

      mov ax, num1
      imul num2  ; ax * num2

      mov WORD PTR result, ax
      mov WORD PTR result+2, dx.

      mov eax, result.
      Call WriteInt.
      Call CrLF

      exit.
  main ENDP
  END main.
```

Question 9:

```
Include Irvine32.inc

.data
    array WORD 10,60,20,33,72,89,45,65,72,18.
    Sample Word 50
    ArraySize WORD 10
    index WORD 0
    Sum WORD 0

.code
main PROC

    StartLoop:
        mov ax,index
        cmp ax, ArraySize
        jae EndLoop ; index >= ArraySize

        mov bx, index
        Shl bx, 1
        mov ax, array[bx].
        cmp ax, Sample
        ja SkipAd ; ax > sample.

        add sum, ax
        SkipAdd:
            inc index
            jmp StartLoop

    ENDLOOP:
        movzx eax,sum
        call WriteDec
        call CrLf

        exit
main ENDP
END main.
```

Question 10:

$36 = 2^5 + 2^2 = 32 + 4$. $EAX \times 36 = (EAX \times 32) + (EAX \times 4)$

a) Multiply EAX by 26.

$26 = 32 - 1$

$26 = 16 + 8 + 2$.

Shl = Value $\times 2^n$.

$2^n = 16, 8, 2$.

4, 3, 1.

- code.

```
Main PROC.  ; EAX * 26.
1. mov ebx, eax.
2. Shl eax, 4.      ; value x 2^4 : [2^4 = 16].
3. mov ecx, ebx.
4. Shl ecx, 3       ; ecx = ebx * 2^3.
5. add. eax, ecx    ; eax + ecx.
6. Shl ebx, 1       ; ebx = ebx * 2^1.
7. add eax, ebx     ; eax = x * 26.
   Call writeint.
   Call crlf.
   exit
main endp.
END main.
```

b) EAX = 2. ; Ro

$2 * 26 = 52$.

1. eax = 2., ebx = 2.
2. $eax = 2 \times 2^4 = 32$.
3. ecx = ebx = 2.
4. $ecx = 2 \times 2^3 = 16$.
5. eax = eax + ecx.
   = 16 + 32.
   = 48.
6. $2 \times 2 = 4 = ebx$.
7. eax = 48 +
   ebx = 4. = 52.

$\boxed{8.\ eax = 52}$